Project - High Level Design

On

Autonomous Logistic Researcher Agent

Course Name: - Agentic Ai

Medi-Caps University – Datagami Skill Based Course

Sr no	NAME	ENROLMENT NUMBER
1	Sarthak Dubey	EN22CS301876
2	Samrath Singh	EN22CS301855
3	Sangini Agrawal	EN22CS301863
4	Shivatimika Pawar	EN22CS301918
5	Shivam Kushwaha	EN22CS301904
6	Satyam Patel	EN22CS301885

Group Name: - 10D8

Project Number: - AAI-45

Industry Mentor: -

University Mentor: -

Academic Year: - 2025-2026

Table Of Content

		Page no.
Section 1	Introduction	3
	1.1 Scope of Document	3
	1.2 Intended Audience	4
	1.3 System Overview	5
Section 2	System Design	6
	2.1 Application Design	6
	2.2 Process flow	7
	2.3 Information flow	7
	2.4 Component Design	8
	2.5 Key Design Consideration	9
	2.6 Api catalogue	9
Section 3	Data Design	10
	3.1 Data model	10
	3.2 Data access mechanism	11
	3.3 Data retention policies	12
	3.4 Data migration	12
Section 4	Interfaces	13
	4.1 User Interface UI	13
	4.2 Backend Api interface	13
	4.3 Internal Agent interface	14
Section 5	State and session management	15
Section 6	Caching	16
Section 7	Non-Functional Requirements	17
	7.1 Security Aspects	17
	7.2 Performance Aspects	17
Section 8	References	19

1. Introduction

The logistics and warehousing industry generates a massive amount of operational data on a daily basis. Each processed order produces structured information such as warehouse location, product type, quantity, delivery distance, transport mode, picking time, and delay status. While this data contains valuable insights about operational efficiency and supply chain performance, most organizations still rely on manual analysis to interpret it. As described in the project overview

Autonomous-Logistics-Researcher- this dependence on human analysts makes it difficult to quickly identify delay causes, performance bottlenecks, or improvement opportunities, ultimately slowing down decision-making.

To overcome these limitations, the Autonomous Logistics Research Agent introduces an AI-driven multi-agent system capable of understanding natural language queries and converting them into actionable insights. Instead of functioning as a simple chatbot, the system behaves like an intelligent logistics analyst. It can retrieve and analyze warehouse data, perform industry research, generate structured reports, and recommend operational improvements. The system is designed to reduce manual effort while improving accuracy and response time in logistics decision-making.

A key architectural principle of the system is the integration of Retrieval Augmented Generation (RAG). This approach ensures that responses are grounded in real data rather than purely generated text. When a user asks a question related to warehouse performance, the system retrieves relevant dataset records before generating a response. Similarly, for research-based queries, it gathers contextual information before synthesizing insights. This significantly reduces hallucination and improves reliability, which is critical in operational environments.

Technically, the system is built using Python, FastAPI, CrewAI, LangChain, ChromaDB, and the Llama-3 Large Language Model via Groq API. These technologies are integrated into a layered architecture that separates presentation, logic, intelligence, and data storage. This modular design ensures scalability, maintainability, and future extensibility. Overall, the Autonomous Logistics Research Agent represents a shift from traditional data reporting systems toward an intelligent, autonomous decision-support platform for modern warehouse operations.

1.1 Scope of the Document

This High-Level Design (HLD) document defines the overall architectural structure and operational framework of the Autonomous Logistics Research Agent. The purpose of this section is to clearly outline what aspects of the system are addressed within this document and to establish the boundaries of its coverage. The document focuses on presenting a comprehensive architectural view of the system, explaining how major components are structured, how they interact, and how they collectively achieve the objective of autonomous logistics analysis.

The scope of this document includes a detailed explanation of the following architectural areas:

- System Architecture Overview – A complete description of how the presentation layer, backend layer, intelligence layer, and data layer are organized and interconnected.
- Multi-Agent Design Model – Explanation of how the Manager Agent, Data Analyst Agent, and Research Agent collaborate within the CrewAI framework.
- Data Handling and Storage Mechanism – Description of dataset processing, embedding generation, and vector storage using ChromaDB.
- API Structure and Communication Flow – Overview of FastAPI endpoints and how user queries are processed internally.
- Memory and Retrieval Strategy (RAG Implementation) – Explanation of how Retrieval Augmented Generation ensures factual and context-aware responses.
- Non-Functional Requirements – Coverage of performance expectations, scalability considerations, modularity principles, and security measures.

Each of these areas is explained not only in terms of functionality but also in terms of architectural reasoning. The document clarifies why specific technologies such as FastAPI, CrewAI, LangChain, ChromaDB, and Llama-3 were selected and how they contribute to system reliability and extensibility. The emphasis is on understanding the *design logic* behind the system rather than just listing its components.

1.2 Intended Audience

This document is intended for individuals who need a clear architectural understanding of the Autonomous Logistics Research Agent without diving into low-level implementation details. It is written to ensure that both technical and non-technical stakeholders can understand how the system is structured, how it operates, and how it can be scaled or extended in the future.

The primary audience includes:

- **Project Evaluators and Academic Reviewers**, who require a structured overview of system architecture and design decisions.
- **System Architects and Developers**, who need clarity on component interactions and architectural flow before implementation or enhancement.
- **AI and Data Science Enthusiasts**, who are interested in understanding how multi-agent systems and Retrieval Augmented Generation (RAG) are applied in a real-world logistics use case.
- **Logistics and Operations Stakeholders**, who want to understand how the system supports warehouse performance analysis and decision-making.

The content is structured to remain concise, technically accurate, and accessible, ensuring that readers from different backgrounds can interpret the system design effectively.

1.3 System Overview

The Autonomous Logistics Research Agent is an AI-powered multi-agent system designed to analyze warehouse operations and perform logistics research through natural language interaction. Unlike traditional reporting systems that only display stored data, this system interprets user queries, performs analysis or research based on the intent, and generates structured, decision-ready responses. Its core objective is to act as an intelligent digital logistics analyst capable of reducing manual effort and improving operational decision-making.

The system is built using a layered architectural model, where each layer has a specific responsibility:

- **Presentation Layer** – Allows users to enter queries through a simple interface.
- **Backend Layer (FastAPI)** – Receives requests, validates them, and routes them internally.
- **Intelligence Layer (CrewAI + LangChain + LLM)** – Manages agent coordination, reasoning, and response generation.
- **Data Layer (CSV + ChromaDB)** – Stores structured warehouse data and vector-based knowledge memory.

A key architectural feature of the system is its multi-agent design. Instead of relying on a single AI model, responsibilities are distributed among specialized agents:

- The **Manager Agent** interprets the query and assigns tasks.
- The **Data Analyst Agent** processes warehouse datasets and identifies operational insights.
- The **Research Agent** gathers and summarizes external logistics knowledge.

The system also integrates **Retrieval Augmented Generation (RAG)** to ensure responses are grounded in factual data. Before generating a final answer, relevant information is retrieved from either the dataset or the vector database. This significantly improves reliability and reduces hallucination, making the system suitable for operational decision support.

2. System Design

The System Design section explains the structural organization of the Autonomous Logistics Research Agent. The system follows a layered and modular architecture to ensure scalability, maintainability, and controlled AI reasoning. Each subsection below describes a specific architectural perspective supported by structured diagrams for clarity.

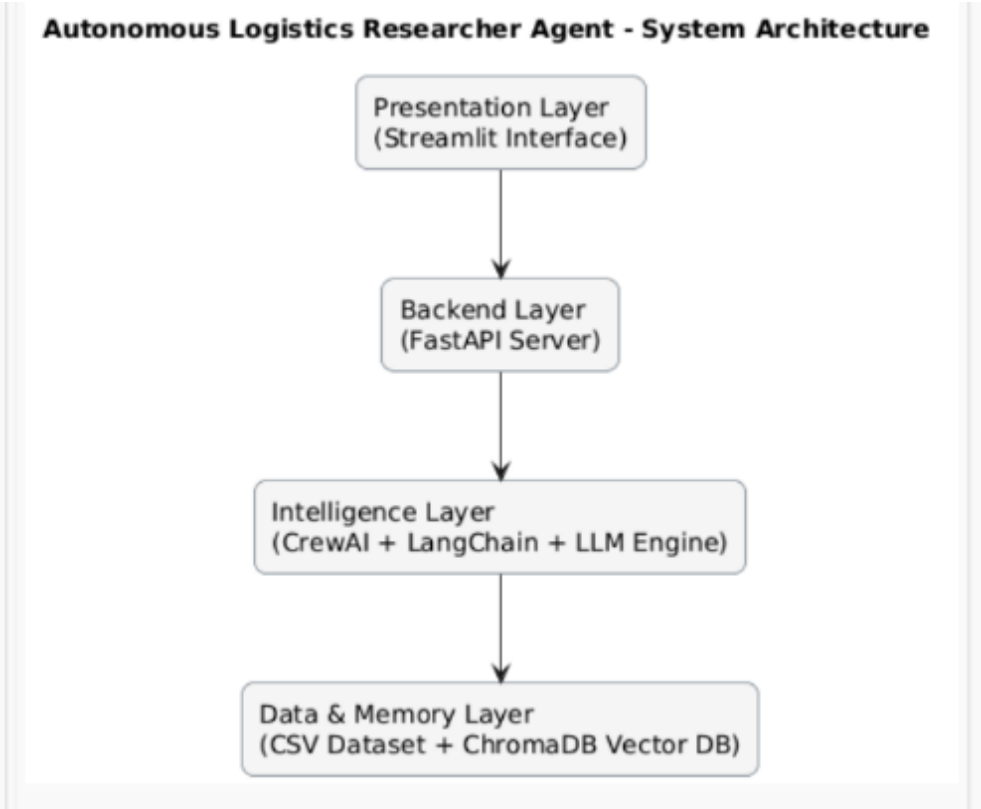
2.1 Application Design

The application is designed using a layered architecture model. Each layer has a clearly defined responsibility to prevent tight coupling and to allow independent upgrades.

The architecture is divided into four major layers:

- Presentation Layer
- Backend Layer
- Intelligence Layer
- Data & Memory Layer

Application Architecture Diagram



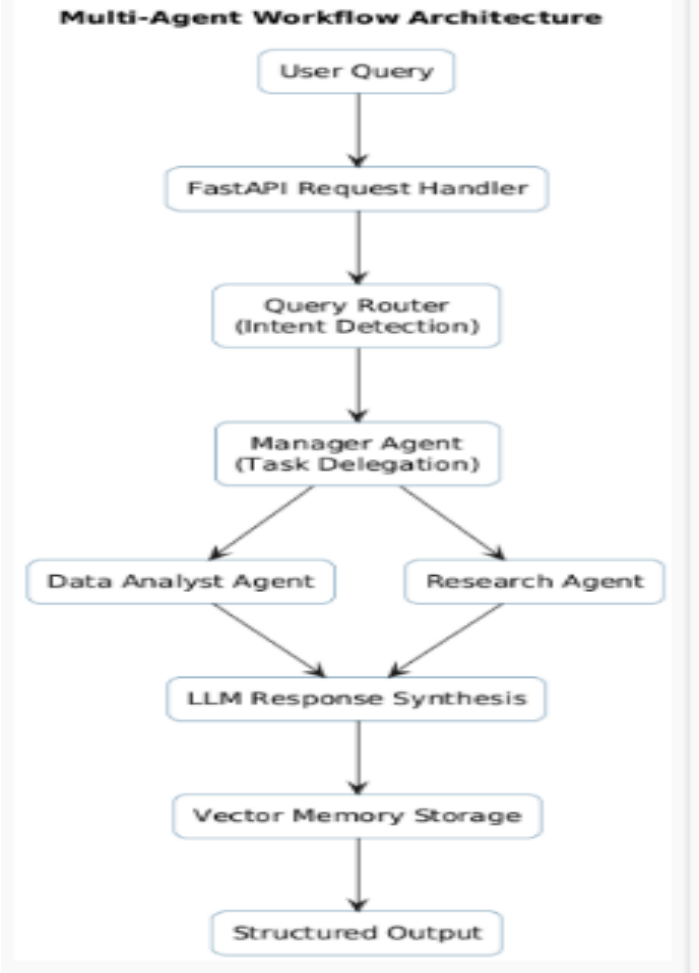
The Presentation Layer handles user interaction.
The Backend Layer validates and routes requests.

The Intelligence Layer performs reasoning and agent orchestration.
The Data Layer manages structured storage and semantic memory.

This separation ensures clean architectural boundaries and simplifies system expansion.

2.2 Process Flow

The process flow describes the lifecycle of a query from submission to response generation. The system follows a staged pipeline rather than direct LLM invocation to ensure controlled reasoning.

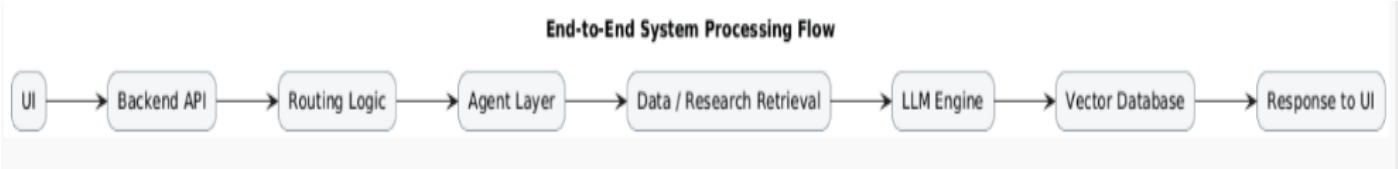


The process begins with user input and ends with structured output. The Manager Agent ensures correct delegation, while the LLM synthesizes the final response after contextual retrieval.

2.3 Information Flow

Information flow explains how data moves internally between components during execution. The system uses a one-directional staged flow to maintain traceability and avoid circular dependencies.

Information Flow Diagram



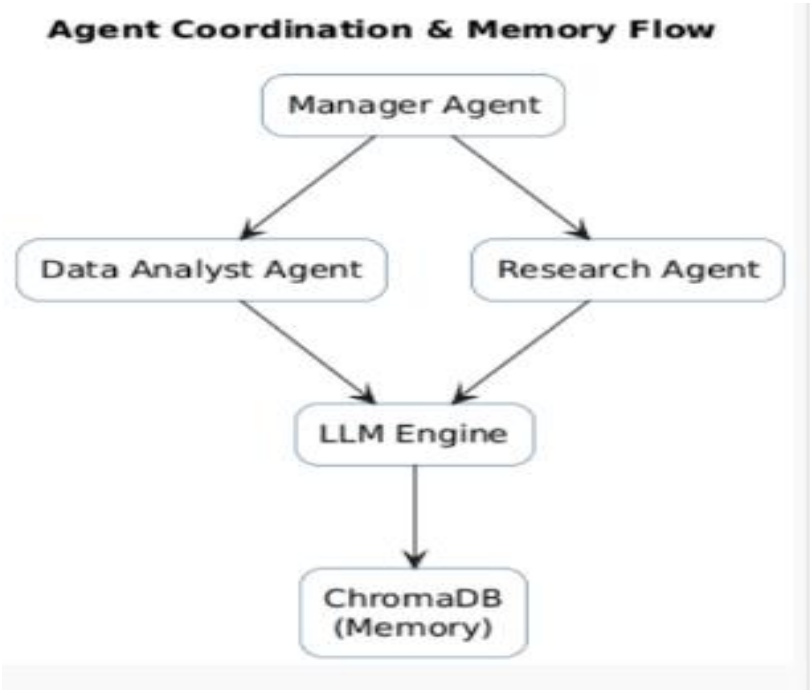
Each stage passes enriched information to the next stage. The system avoids direct interaction between UI and LLM, ensuring structured control over response generation.

2.4 Component Design

The system consists of five core components:

- Manager Agent
- Data Analyst Agent
- Research Agent
- LLM Engine
- Memory System

Component Interaction Diagram



The Manager Agent controls delegation.
Specialized agents perform data collection.
The LLM synthesizes structured output.
The Memory layer stores embeddings for RAG.

This modular interaction model supports scalability and controlled reasoning.

2.5 Key Design Considerations

The system design was guided by architectural principles that ensure long-term reliability.

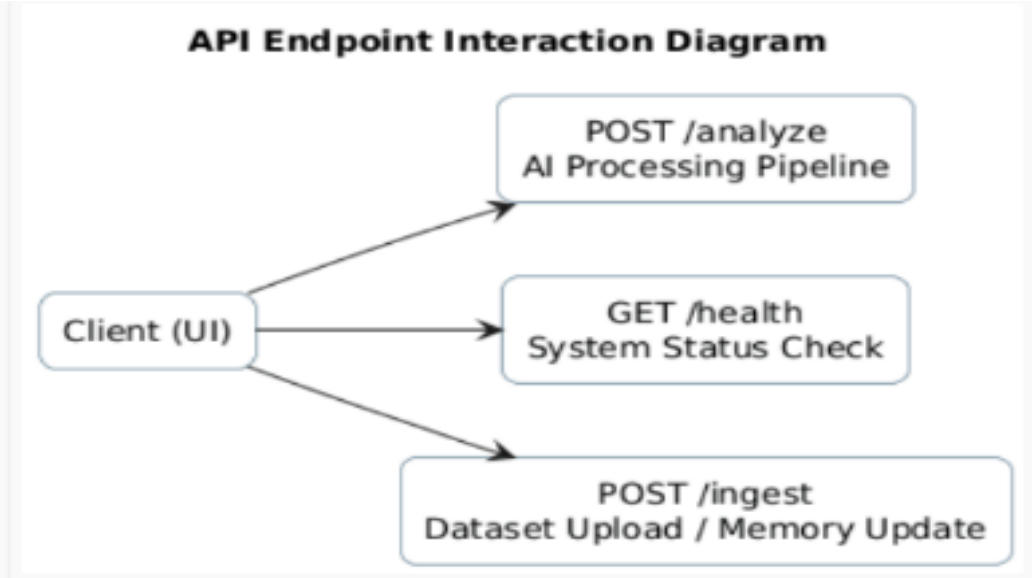
Modularity ensures independent upgradability of components.
Scalability allows additional agents to be integrated without structural redesign.
Performance efficiency is achieved by reducing unnecessary LLM calls.
Reliability is ensured through Retrieval Augmented Generation (RAG), which grounds responses in real data before synthesis.

These considerations ensure that the system is not only functional but also enterprise-ready.

2.6 API Catalogue

The system exposes RESTful APIs through FastAPI to manage communication between frontend and backend.

API Structure Diagram



- **POST /analyze** accepts a user query and returns a structured AI-generated response.
- **GET /health** verifies backend service availability.
- **POST /ingest** allows dataset or knowledge ingestion into memory.

All APIs operate using JSON request-response format and remain stateless, ensuring scalability and ease of deployment.

3. Data Design

The Data Design section defines how data is structured, stored, accessed, and maintained within the Autonomous Logistics Research Agent. Since the system operates on both structured operational data and semantic memory data, the architecture supports two different forms of storage: tabular dataset storage and vector-based knowledge storage. This hybrid design ensures precise analytical capability along with contextual retrieval support for intelligent response generation.

3.1 Data Model

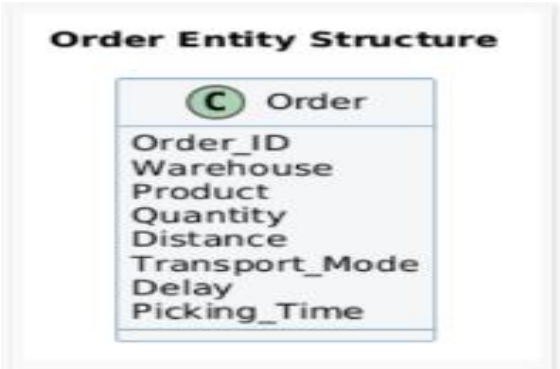
The data model represents the structured format in which warehouse operational information is stored and processed. For this system, the dataset is maintained in a tabular structure (CSV format), where each row corresponds to a single order transaction and each column represents a specific operational attribute.

The primary attributes included in the dataset are:

- Order ID
- Warehouse Location
- Product Category
- Quantity
- Distance (in kilometers)
- Transport Mode
- Delay Status
- Picking Time (in minutes)

These attributes were selected because they directly influence warehouse efficiency and delivery performance. For example, delivery distance and transport mode affect transit time, while picking time reflects internal warehouse processing efficiency. The delay status helps in identifying bottlenecks and correlating performance patterns.

The relationship between these entities can be conceptually visualized as follows:



Each order acts as a central entity containing attributes that collectively determine operational performance. This simple but effective relational model allows deterministic filtering and analysis by the Data Analyst Agent.

In addition to structured tabular data, the system also maintains semantic data in the form of vector embeddings stored inside ChromaDB. These embeddings represent previously generated reports and research insights, enabling similarity-based retrieval for future queries.

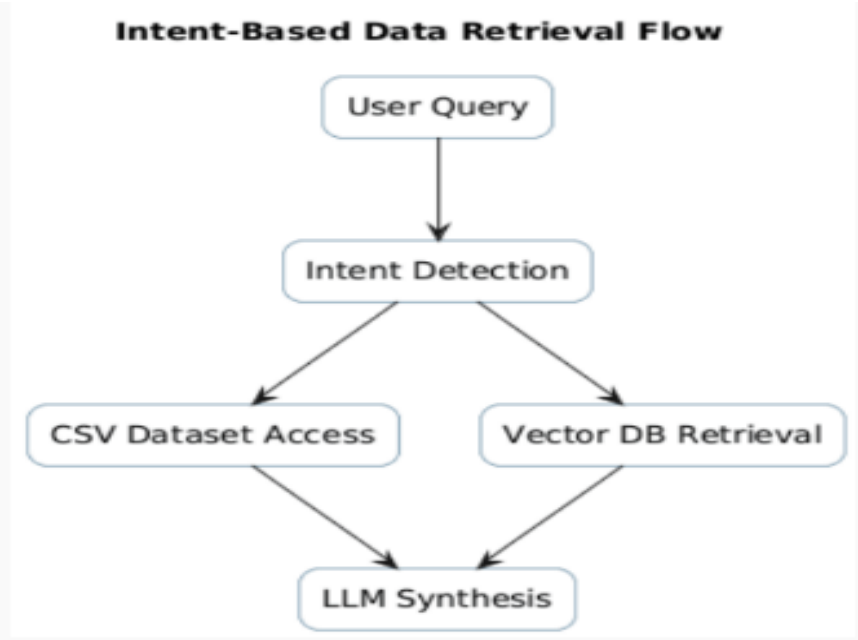
3.2 Data Access Mechanism

The Data Access Mechanism describes how the system retrieves and processes information during query execution. Since the system supports both structured and semantic data retrieval, it follows two access paths depending on query type.

For structured dataset queries, the Data Analyst Agent loads the CSV file into memory using data processing libraries. It then performs filtering, aggregation, and pattern detection operations based on the query intent. This ensures deterministic and controlled data handling without relying solely on the language model.

For research or contextual queries, the system retrieves relevant embeddings from the ChromaDB vector database. These embeddings are selected using similarity search techniques. The retrieved contextual information is then injected into the LLM prompt to ensure grounded response generation.

The overall access mechanism can be illustrated as:



This dual-path design ensures efficient processing while maintaining contextual accuracy.

3.3 Data Retention Policies

Data retention policies define how long information is stored and how it is managed over time. Since the system operates primarily as an analytical assistant rather than a transaction-processing system, it does not modify original warehouse data. The structured CSV dataset remains static unless manually updated.

However, the vector memory layer continuously stores embeddings of generated reports and research insights. These embeddings are retained to improve future query responses. Over time, this creates an evolving knowledge base.

Retention policies include:

- Structured dataset remains unchanged unless re-ingested.
- Generated knowledge embeddings are stored for long-term reuse.
- Logs and API request records may be retained for debugging and monitoring.
- Memory pruning mechanisms can be introduced in the future to remove outdated insights.

The retention strategy prioritizes knowledge accumulation while maintaining system stability.

3.4 Data Migration

Data migration refers to the process of transitioning the system from prototype-level storage to production-grade infrastructure. Currently, the system uses CSV for structured storage and local ChromaDB for vector memory. While this is sufficient for academic or small-scale deployment, scalability requires planned migration strategies.

The migration roadmap includes:

- Moving from CSV to relational databases such as PostgreSQL for handling larger datasets.
- Migrating local ChromaDB storage to cloud-based vector databases for distributed access.
- Containerizing the system using Docker for deployment portability.
- Integrating cloud storage solutions for secure and scalable data hosting.

4. Interfaces

The Interfaces section describes how different components of the Autonomous Logistics Research Agent communicate with each other and how external entities interact with the system. Since the system follows a layered architecture, interfaces exist at multiple levels: between the user and the system, between frontend and backend, and internally between backend and AI modules.

The system primarily exposes interfaces through RESTful APIs, enabling communication in a standardized and platform-independent format. These interfaces ensure smooth data exchange while maintaining modularity and scalability.

4.1 User Interface (UI)

The User Interface acts as the entry point for system interaction. It allows users to submit natural language queries and view structured responses generated by the AI system.

The UI performs the following responsibilities:

- Captures user input.
- Sends HTTP requests to backend APIs.
- Displays structured reports including insights and recommendations.
- Handles basic validation and user feedback.

The interface is intentionally designed to remain simple and business-friendly. Users are not required to understand AI agents, databases, or backend processing logic. The system abstracts technical complexity behind an intuitive interaction layer.

4.2 Backend API Interface

The Backend API interface is implemented using FastAPI and serves as the communication bridge between the frontend and intelligence layer. All user queries are transmitted in JSON format to the backend server.

The backend interface ensures:

- Request validation.
- Error handling.
- Query routing.
- Secure communication between layers.

The backend exposes REST endpoints such as query processing, health monitoring, and dataset ingestion. This design allows integration with other external systems if required in the future.

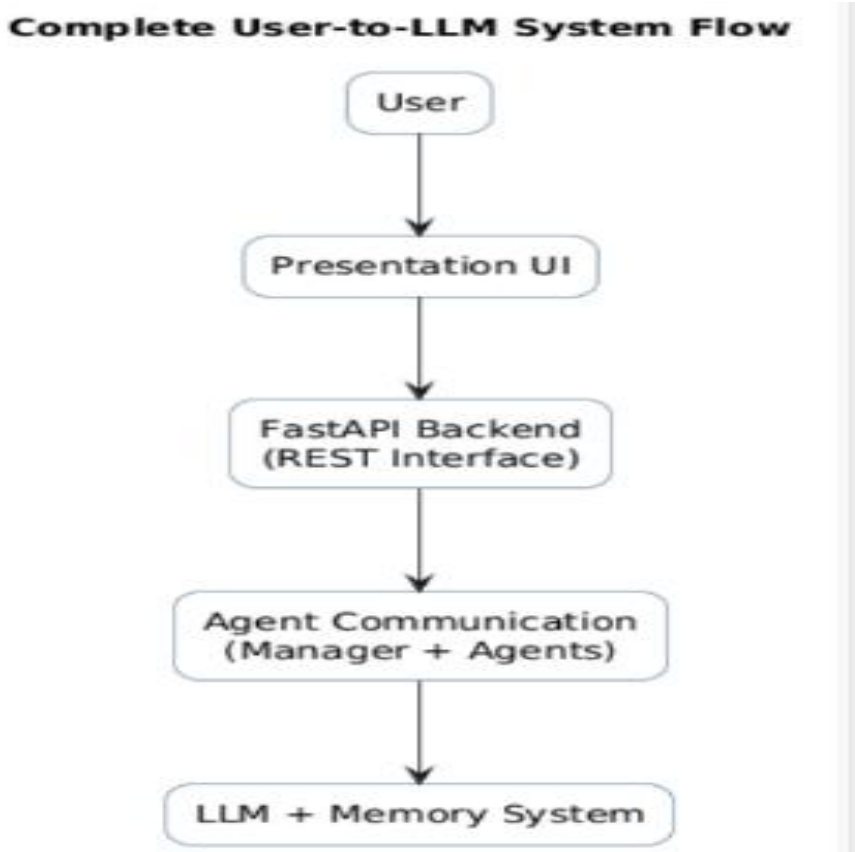
4.3 Internal Agent Interface

Within the system, internal interfaces exist between the Manager Agent, specialized agents, the LLM engine, and the memory layer. These interfaces are not exposed externally but are critical for modular communication.

Each agent communicates through structured prompts and controlled data exchange rather than raw text passing. This ensures that:

- Data passed to the LLM is contextualized.
- Agents remain loosely coupled.
- Execution remains traceable.

Interface Interaction Diagram



This diagram illustrates how external communication flows through REST APIs, while internal communication flows through controlled agent interactions.

5. State and Session Management

State and session management define how the system handles user interactions over time and how contextual information is preserved during execution. The Autonomous Logistics Research Agent is primarily designed as a stateless API-driven system, meaning each user request is processed independently without relying on previous request states at the server level.

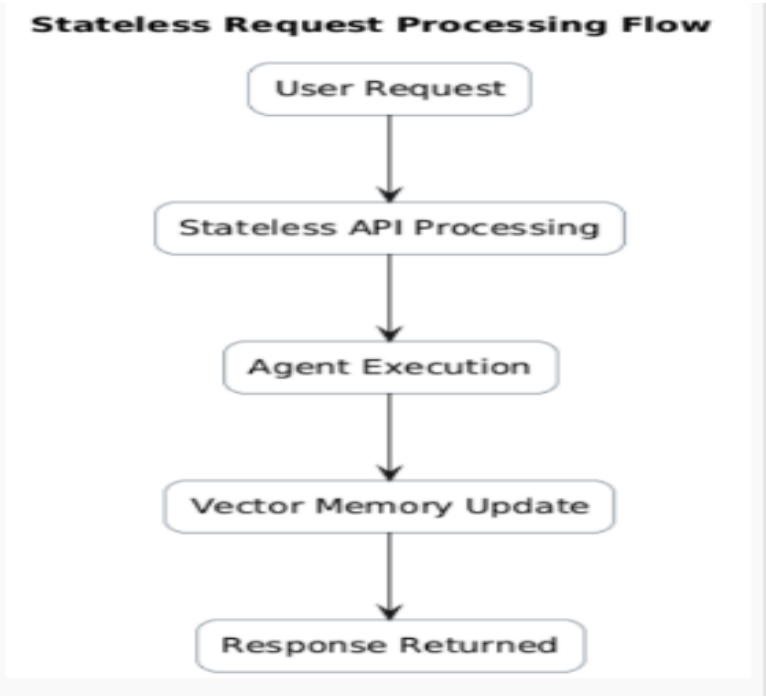
In a stateless architecture, the backend does not permanently store session-specific information such as user identity or conversation flow. Every query sent to the FastAPI server is treated as a new request. This design improves scalability because the server does not need to maintain memory for multiple concurrent users. It also simplifies deployment in distributed or cloud-based environments, where multiple instances of the application may run simultaneously.

Although the backend is stateless, the system maintains contextual intelligence through its vector memory layer. Instead of storing conversational sessions traditionally, the system stores semantic embeddings of generated responses inside ChromaDB. When a similar query is asked in the future, relevant past knowledge is retrieved using similarity search. This approach provides contextual continuity without maintaining active session objects.

The distinction between session state and knowledge state is important. Session state refers to temporary data maintained during a user interaction, while knowledge state refers to stored insights that can be reused across interactions. In this system:

- Session state is minimal and request-based.
- Knowledge state is persistent and stored in the vector database.

The session handling flow can be visualized as:



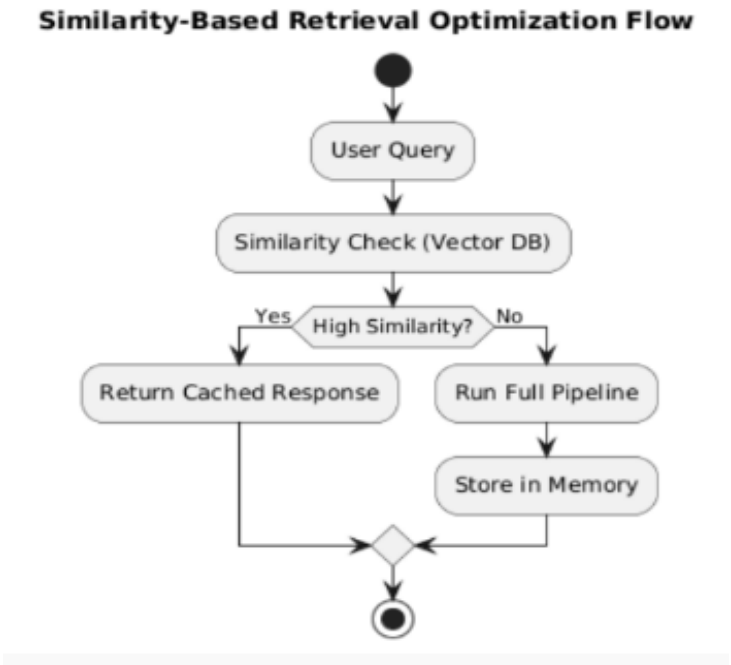
6. Caching

Caching plays an important role in improving system performance and reducing unnecessary computational overhead in the Autonomous Logistics Research Agent. Since the system involves language model processing and vector similarity search, repeated execution of identical or similar queries can increase response time and operational cost. The caching mechanism is introduced to optimize efficiency while maintaining response accuracy.

The system primarily benefits from semantic caching rather than traditional key-value caching. When a user submits a query, the system first checks the vector memory layer to determine whether a similar query has already been processed. If relevant embeddings are found with high similarity, the stored structured response can be reused or used as contextual grounding for a faster answer. This reduces repeated LLM calls and minimizes API usage costs.

In addition to semantic caching, basic request-level caching can be implemented at the backend layer. If identical queries are received within a short time window, the backend can temporarily store and return previously generated results without triggering the full agent orchestration pipeline. This improves response time and reduces processing load during high-traffic scenarios.

The caching workflow can be illustrated as follows:



This structured caching strategy ensures that the system remains efficient while maintaining contextual accuracy. By prioritizing similarity-based retrieval before full execution, the architecture reduces redundant computation and enhances scalability.

In future deployments, advanced caching mechanisms such as distributed caching or in-memory stores (e.g., Redis) can be integrated to support large-scale enterprise environments. However, for the current system scope, vector-based semantic caching provides a sufficient and intelligent optimization layer.

7. Non-Functional Requirements

Non-functional requirements define the quality attributes of the Autonomous Logistics Research Agent. While functional requirements describe what the system does, non-functional requirements define how well the system performs those functions. In AI-driven systems, non-functional aspects such as security, performance, scalability, and reliability are critical because they directly impact usability, operational trust, and system sustainability.

This section focuses on two primary non-functional dimensions: Security Aspects and Performance Aspects.

7.1 Security Aspects

Security is a fundamental requirement for any system that processes operational data and interacts with external APIs. Although the current implementation is designed as a prototype-level intelligent system, its architecture incorporates foundational security considerations to ensure safe and controlled execution.

One important security measure is the protection of API credentials. Since the system integrates external services such as a Large Language Model API, authentication keys are stored in environment variables rather than hardcoded into the application. This reduces the risk of credential leakage and allows secure configuration management during deployment.

Input validation is another critical security component. Every query received through the FastAPI backend undergoes validation to prevent malformed or malicious input from entering the processing pipeline. This reduces the risk of injection attacks, unexpected runtime failures, or misuse of the API layer.

The system also follows a controlled interaction model with the language model. Instead of directly forwarding user input to the LLM, the query is first routed, analyzed, and enriched with contextual data. This layered approach prevents prompt manipulation and ensures structured processing.

From a deployment perspective, the architecture supports secure communication via HTTPS when deployed in production environments. Role-based access control and authentication mechanisms can also be integrated in future iterations to restrict access to authorized warehouse managers or analysts.

Overall, the system design prioritizes credential safety, controlled input handling, structured LLM interaction, and future extensibility for enterprise-level security compliance.

7.2 Performance Aspects

Performance is critical for maintaining user trust and operational efficiency. Since the system integrates agent orchestration and LLM-based reasoning, response time optimization is carefully considered in the architecture.

One of the primary performance strategies is minimizing unnecessary LLM calls. Instead of sending every query directly to the language model, the system first determines the intent and retrieves relevant structured or semantic data. Only after contextual enrichment does the LLM generate a response. This reduces computational cost and improves response speed.

The use of vector similarity search through ChromaDB also enhances performance. By retrieving contextually relevant knowledge before synthesis, the system avoids redundant processing and accelerates repeated query handling. This mechanism acts as both a memory system and an optimization layer.

The backend is implemented using FastAPI, which supports asynchronous request handling. This enables the system to manage multiple concurrent requests efficiently without blocking execution. Such asynchronous processing improves scalability under higher query loads.

Performance optimization is also supported by caching mechanisms, where similar or identical queries can reuse previously generated results. This further reduces latency and API consumption.

In its current configuration, the system is optimized for moderate-scale usage, such as academic demonstrations or controlled enterprise environments. However, the modular architecture allows performance scaling through database migration, distributed deployment, or integration with cloud infrastructure when needed.

8. References

The design and implementation of the Autonomous Logistics Research Agent are based on established AI frameworks, web technologies, and research concepts. The following references provided technical and theoretical support for the system architecture and development:

1. CrewAI Documentation – Multi-agent orchestration framework.
2. LangChain Documentation – LLM integration, tool usage, and memory management.
3. ChromaDB Documentation – Vector database and semantic retrieval mechanisms.
4. FastAPI Documentation – Backend API development and asynchronous processing.
5. Retrieval-Augmented Generation (RAG) Research Paper by Lewis et al. – Conceptual foundation for combining retrieval with generative AI.
6. Groq API and Llama-3 Documentation – Large Language Model integration and inference handling.

These references collectively support the architectural decisions, agent-based design, and Retrieval Augmented Generation strategy implemented in this project.